

Benchmarking GNN Inference on the Intel Core Ultra NPU: A Latency, Quantization, and Energy Analysis

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

Abstract—Neural processing units (NPUs) in client systems-on-chip (SoCs) target low-power inference, but their behavior on sparse graph neural networks (GNNs) remains underexplored. We characterize 14 GNN, CNN, and transformer models across the CPU, integrated GPU (iGPU), and NPU of an Intel Core Ultra platform using OpenVINO 2024.1. GNN experiments use fixed-shape graph inputs derived from three Open Graph Benchmark datasets. The NPU provides substantial CPU-relative speedups for selected dense FP32 models, including MobileNetV2 ($4.5\times$), ResNet-50 ($8.0\times$), and ViT-Tiny ($11.4\times$), whereas it provides little benefit over the CPU for most evaluated GNNs and the iGPU achieves the lowest latency for most GNN configurations. INT8 behavior is model-dependent: several models show marginal gains, SGC regresses by $2.2\times$, and three architectures yield no executable INT8 configuration. Package-level measurements for selected CPU and iGPU configurations fall within 9–12.5 W; direct NPU power isolation is unavailable. These results show that backend selection depends strongly on model structure, quantization compatibility, and verified device assignment. The suite is available at <https://yusufarbc.github.io/intel-npu-gnn-benchmarking/>.

Index Terms—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, quantization, benchmarking, edge inference.

I. INTRODUCTION

Neural processing units (NPUs) in client systems-on-chip (SoCs) are specialized hardware accelerators engineered for low-power, energy-efficient deep learning inference. In the Intel Meteor Lake processor architecture (e.g., Core Ultra 5 125H), the AI Boost NPU operates as a dedicated low-power engine alongside the multi-core CPU and integrated GPU (iGPU) within a nominal 28 W mobile client form factor. While such consumer-tier client SoCs are optimized for computer vision workloads, their practical performance on sparse, irregular graph neural networks (GNNs) remains underexplored relative to high-power datacenter accelerators.

Unlike dense tensor operations, GNN execution combines sparse-times-dense matrix multiplication (SpDMM), often called SpMM in the GNN literature, with dynamic feature gathering over irregular adjacency structures, yielding high sensitivity to data movement [1], [2]. As shown in Fig. 3, the exported GNN graphs contain $2\text{--}4\times$ more shape-manipulation

operators (Gather, Scatter, Reshape) than the vision graphs. These operators involve irregular index dependencies that are less regular than the dense tensor workloads targeted by the NPU’s streaming-dataflow architecture. While prior works evaluate GNNs on server GPUs and custom ASICs [3]–[5], consumer client NPUs present a different design point: a restricted power envelope, shared system memory, compiler-managed data movement, and incomplete support for sparse or dynamic primitives [6], [7].

We develop a reproducible benchmarking suite evaluating 14 models across three Open Graph Benchmark (OGB) datasets and three Meteor Lake backends (CPU, iGPU, NPU) under OpenVINO 2024.1. The contribution is a boundary characterization of an accessible client NPU: it identifies where nominal accelerator availability translates into native execution, where quantization regresses, and where toolchain fallback can invalidate device-level comparisons. Our contributions include:

- A reproducible profiling protocol (100 iterations $\times$ 3 runs $\times$ 3 datasets) extracting latency, throughput, precision gain, compilation outcomes, and ONNX Runtime execution traces.
- Cross-architectural latency comparisons showing that the iGPU achieves the lowest latency for most evaluated GNNs, whereas the NPU provides its largest CPU-relative gains on selected dense FP32 models.
- Empirical evidence showing INT8 quantization provides minimal or negative returns on the NPU for GNNs, with two attention-based architectures yielding no executable INT8 configuration.
- Analysis of operator composition, execution anomalies, and graph-size scaling under static-shape NPU compilation on a client platform.

II. BACKGROUND

A. GNN Computation and Acceleration Landscape

Graph neural network architectures compute node representations via iterative neighborhood aggregation. Across

spectral [8], [9], spatial [10], [11], attentional [12]–[14], and propagation-based [15], [16] models, a central computational primitive is sparse-times-dense matrix multiplication (SpDMM), combined with nonlinear feature transformations. These operations exhibit irregular memory access patterns and poor spatial locality, stressing memory bandwidth and on-chip caches [1], [2], [17].

To alleviate these memory bottlenecks, domain-specific GNN accelerators such as HyGCN [3], EnGN [1], GRIP [4], and TT-GNN [5] implement dedicated hardware units for irregular edge gathering, custom scratchpad hierarchies, and crossbar interconnects. However, these custom ASICs require specialized non-commodity silicon. In contrast, commodity edge accelerators and client NPUs [18], [19] are designed around dense matrix multiplication engines optimized for regular systolic dataflows.

B. Intel Meteor Lake NPU Architecture

Intel Meteor Lake client SoCs integrate the AI Boost NPU on the SoC tile and share system memory with the CPU and iGPU across the SoC fabric [6], [7]. This client architecture makes end-to-end performance sensitive to data movement. Its streaming execution model favors continuous, predictable memory layouts (e.g., CNNs) but can incur overhead on irregular graph topologies with low data reuse. OpenVINO 2024.1.0 introduced a preview NPU plugin for Intel Core Ultra processors [20]. In the evaluated ONNX Runtime/OpenVINO path, unsupported operations may remain on the host CPU. Requested-device labels and registered-provider lists alone do not establish per-operator placement, so we distinguish confirmed compilation outcomes from timing-based execution anomalies.

C. Quantization and Edge Benchmarking

Low-precision integer quantization (INT8) is standard practice for accelerating edge inference [21]. However, quantizing GNNs is complicated by high dynamic activation ranges and non-uniform degree distributions across graph nodes [22]. Modern graph compilers apply operator fusion to minimize memory traffic [23], [24], but sparse indexing patterns frequently break fusion chains. MLPerf Inference covers inference systems, while GNNMark characterizes GNN training on GPUs [25], [26]; neither benchmark targets GNN inference on a Meteor Lake-class client NPU. Empirical evaluation with real-world graph datasets [27] therefore remains an open area of research.

III. METHODOLOGY

A. Experimental Platform

Experiments were conducted on an Intel Core Ultra 5 125H mobile processor (nominal 28 W processor base power, up to 4.50 GHz maximum turbo frequency, 16 GB LPDDR5x RAM, Windows 11 Build 26200) [28]. This is a power-constrained client platform rather than a server- or datacenter-class accelerator. The evaluated backends are: (1) the 14-core CPU (4 Performance, 8 Efficient, and 2 Low-Power

Efficient cores), (2) the integrated Arc GPU (iGPU, Xe-LPG architecture, 7 Xe-cores), and (3) the AI Boost NPU (3720 series, VPUX37XX microarchitecture). OpenVINO’s device identifier GPU refers to this integrated GPU; we use *iGPU* throughout the paper. The software stack uses OpenVINO 2024.1 and ONNX Runtime 1.18.

B. Benchmarked Models

Table I lists the 14 evaluated models. The GNN selection covers spectral (GCN, SGC), spatial (GraphSAGE, GIN), attentional (GAT, GATv2), propagation (APPNP), hybrid (GraphTransformer), and message-passing (MPNN) paradigms. These models (0.02M–0.20M parameters) represent edge-deployment scales where memory limits constrain size. Baselines include CNNs (ResNet-50, MobileNetV2, EfficientNet-B0) and transformers (ViT-Tiny, BERT-Tiny). All models were exported to ONNX at FP32. INT8 artifacts were generated through the mixed conversion pipeline described below; GAT and GATv2 did not produce executable INT8 configurations and are evaluated in FP32 only.

TABLE I
EVALUATED MODELS AND CHARACTERISTICS

Model	Category	Params	INT8
<i>Graph Neural Networks</i>			
GCN [8]	Spectral	0.10M	✓
GAT [12]	Attentional	0.11M	×
GATv2 [13]	Attentional	0.13M	×
GIN [11]	Expressive	0.09M	✓
GraphSAGE [10]	Spatial	0.18M	✓
SGC [9]	Simplified	0.02M	✓
APPNP [15]	Propagation	0.09M	✓
GraphTransformer [14]	Hybrid	0.18M	✓
MPNN [16]	Message-Passing	0.20M	✓
<i>Dense Baselines</i>			
ResNet-50 [29]	CNN	25.5M	✓
MobileNetV2 [30]	CNN	3.5M	✓
EfficientNet-B0 [31]	CNN	5.3M	×
ViT-Tiny [32]	Vision Trans.	5.7M	×
BERT-Tiny [33]	NLP Trans.	4.4M	✓

× = no executable INT8 configuration was produced. ✓ = INT8 benchmarked.

C. Datasets

We use three real-world Open Graph Benchmark (OGB) datasets [27] as graph-data sources (Table II). For compatibility with the statically exported ONNX graphs, each GNN inference uses a fixed input containing 2,708 nodes, 10,000 edges, and 1,433 feature columns. Larger graphs are sampled, while edge lists and feature dimensions are truncated or zero-padded to the exported tensor shapes. Table II therefore describes the source graphs rather than the effective density of the inference tensors, and the measurements do not represent full-graph OGB inference. Vision and NLP models receive synthetic inputs matching native shapes (e.g., $1 \times 3 \times 224 \times 224$ for CNNs); their repeated dataset labels use the same synthetic

input and are not independent dataset workloads. Dataset preprocessing and tensor extraction run once in a separate Python subprocess before model warm-up and timing because the dataset and inference stacks require potentially conflicting native-library environments. The subprocess serializes the prepared tensors to disk and exits; process startup, loading, and serialization are excluded from the reported steady-state inference latency.

TABLE II
OGB DATASET CHARACTERISTICS

Dataset	Nodes	Edges	Features	Edges/node
ogbn-arxiv	169K	1.16M	128	6.9
ogbn-products	2.45M	61.9M	100	25.3
ogbn-proteins	132.5K	39.6M	8	298.5

D. Quantization Methodology

The INT8 conversion pipeline is not uniform across model families. For GNNs, it first attempts NNCF post-training quantization with 300 synthetic samples matching the fixed export shapes and falls back to ONNX Runtime dynamic quantization if NNCF conversion fails. ResNet-50 and MobileNetV2 use NNCF with 50 synthetic image samples, whereas BERT-Tiny and locally exported additional baselines use ONNX Runtime dynamic quantization. The results therefore characterize the generated deployment artifacts rather than a controlled comparison of quantizers. FP32 and INT8 are the two evaluated precisions, not a claim about the complete set supported by the hardware. Post-quantization task accuracy was not evaluated, so Section IV-B reports execution-performance outcomes rather than accuracy–performance trade-offs.

E. Measurement Protocol

Each configuration runs a 5-iteration warm-up and 100 timed iterations, repeated across three independent runs. We report the arithmetic mean and sample standard deviation of the three run means. Where confidence intervals are reported in the artifacts, they are two-sided Student-*t* intervals over the three run means. All optimized runs use ONNX Runtime graph optimizations (ORT_ENABLE_EXTENDED) under the respective OpenVINO or CPU providers.

F. Artifacts and Metrics

We collect end-to-end inference latency (T_{inf} in ms), throughput ($1000/T_{\text{inf}}$ inferences/s), INT8/FP32 speedup ($T_{\text{FP32}}/T_{\text{INT8}}$), compilation status, runtime traces, and graph-optimization speedup ($T_{\text{baseline}}/T_{\text{optimized}}$). Package power is measured via Intel SoCWatch (`-f sys -t 30 -r sum`) for GCN and MPNN on CPU and iGPU backends; NPU-specific power-rail isolation was unsupported by the client platform telemetry.

IV. RESULTS

A. Cross-Architectural Latency Comparison

Inference latency is the mean wall-clock time in milliseconds (T_{inf}) for one batch-size-one forward pass after warm-up. It includes execution-provider dispatch and required host–device transfers, but excludes dataset loading, preprocessing, model conversion, and one-time graph compilation. For sequential batch-size-one execution, the corresponding throughput is $1000/T_{\text{inf}}$ inferences/s; latency is reported because it is the direct responsiveness metric for the targeted interactive edge setting. Fig. 1 compares FP32 latencies across backends. The iGPU achieves the lowest latency for most GNNs, e.g., GraphTransformer (6.03 ms iGPU vs. 10.72 ms NPU vs. 10.69 ms CPU) and APPNP (7.88 ms iGPU vs. 11.75 ms NPU vs. 11.82 ms CPU). GCN reaches 6.94 ms on iGPU and 7.91 ms on NPU. Several GNNs show near parity between NPU and CPU, while GraphSAGE and SGC are exceptions to the broader iGPU-leading trend. Conversely, the NPU provides substantial CPU-relative acceleration for selected dense workloads: MobileNetV2 achieves 1.90 ms ($4.5\times$ over CPU), ResNet-50 runs at 3.94 ms ($8.0\times$), and ViT-Tiny reaches 9.10 ms ($11.4\times$ over CPU). The fastest backend remains model-dependent; the iGPU is faster than the NPU for EfficientNet-B0 and ViT-Tiny.

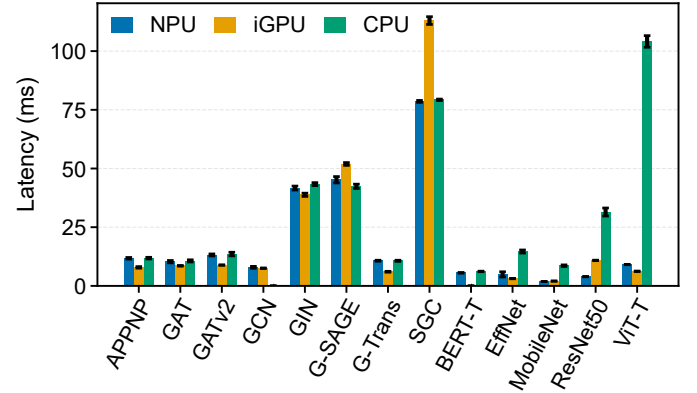


Fig. 1. FP32 latency comparison across CPU, iGPU, and NPU backends. Error bars indicate the sample standard deviation of three run means. Several GNNs show CPU–NPU parity, while selected dense models exhibit substantial NPU speedup over CPU. Missing bars indicate unavailable measurements.

B. INT8 Quantization: Performance Impact

Table III shows INT8 speedups for available requested-NPU measurements. GraphTransformer improves by $1.21\times$, while GCN, GIN, and GraphSAGE show marginal gains. Graph-node composition alone does not isolate the mechanism behind these differences. APPNP ($0.84\times$) and SGC regress; SGC’s INT8 latency (173.90 ms) is $2.2\times$ slower than FP32 (78.59 ms, $0.45\times$ speedup). The experiment does not isolate a single compiler or runtime cause for this regression. Device-assignment outcomes require cautious interpretation. MobileNetV2 INT8 on the requested NPU runs at 5.38 ms versus 1.90 ms in FP32 and is close to its 5.46 ms CPU INT8 latency. This

similarity is an execution anomaly, not conclusive evidence of CPU placement without a retained per-operator assignment trace. EfficientNet-B0 did not produce an executable INT8 configuration.

TABLE III
INT8/FP32 SPEEDUP ON NPU (MEAN ACROSS OGB DATASETS)

Model	FP32 (ms)	INT8 (ms)	Speedup
GraphTransformer	10.72	8.90	$1.21\times$
GCN	7.91	7.58	$1.04\times$
GraphSAGE	45.21	42.95	$1.05\times$
GIN	41.68	40.55	$1.03\times$
APPNP	11.75	13.96	$0.84\times$
SGC	78.59	173.90	$0.45\times$
GAT	10.32	—	— [†]
GATv2	13.19	—	— [†]

Speedup > 1.0 indicates INT8 faster. [†]No executable INT8 configuration was produced. EfficientNet-B0 also failed to produce an executable INT8 configuration. Dense baselines are omitted because their INT8 execution outcomes are incomplete or not conclusively verified on NPU.

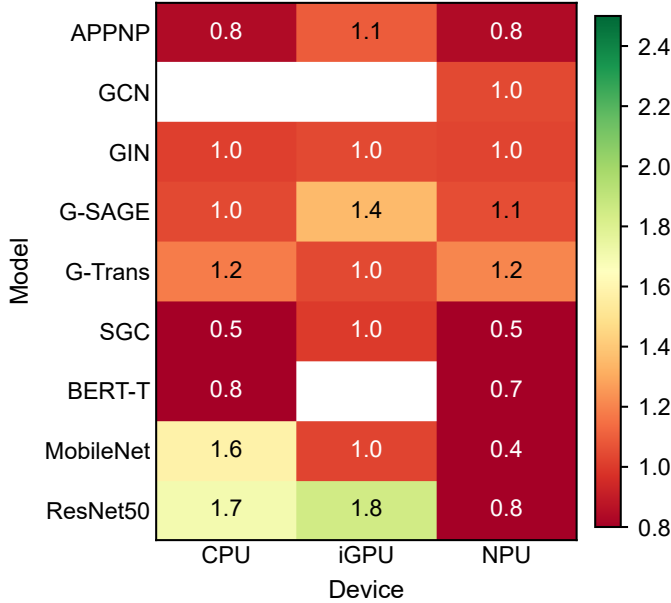


Fig. 2. INT8 speedup heatmap across models and devices. Green values above 1.0 indicate improvement, red values below 1.0 indicate regression, and blank cells indicate unavailable results. Outcomes are model- and device-dependent.

C. Operator Composition Analysis

Fig. 3 reports the fraction of nodes in each exported ONNX graph; it is a structural composition, not an execution-time breakdown. Consequently, a large *Other* segment does not imply that most latency is spent there. *Other* groups inexpensive nonlinear activations (e.g., ReLU, GELU, Sigmoid), normalization, elementwise arithmetic (Add, Sub, Mul), and constant/slicing nodes that would otherwise make the legend unreadable. The relevant contrast is that GNN graphs contain 2–4 $\times$ more Gather, Scatter, Reshape, and Concat nodes than vision graphs, whereas vision graphs contain a larger fraction

of regular Conv nodes. ViT-Tiny (5.7M parameters) achieves an 11.4 $\times$ speedup, whereas GraphTransformer (0.18M parameters) shows no NPU speedup (10.72 ms NPU vs. 10.69 ms CPU). This contrast is consistent with their different operator structures, but graph composition alone does not isolate the cause of the latency difference.

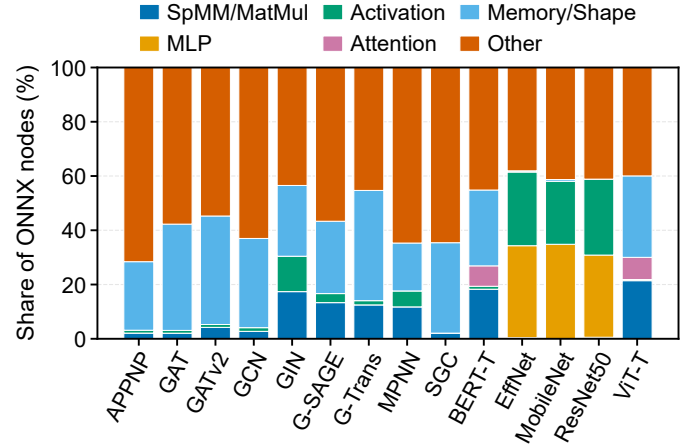


Fig. 3. ONNX graph-node composition (not execution time). *Other* groups activations, normalization, elementwise arithmetic, and constant/slicing nodes; the key contrast is the larger memory/shape fraction in GNNs.

D. Execution and Compilation Outcomes

Table IV replaces the visually uniform fallback heatmap and reports only exceptions that affect interpretation. A requested-device label identifies the target configuration but does not by itself prove per-operator placement. We therefore distinguish unavailable compilations, missing retained native-device measurements, and timing anomalies without converting them into unsupported fallback percentages.

TABLE IV
EXECUTION AND COMPILATION EXCEPTIONS

Model	Configuration	Observed outcome
MPNN	NPU FP32	No retained native-NPU result
MobileNetV2	NPU INT8	CPU-like timing; placement unverified
GAT, GATv2	NPU INT8	No executable configuration
EfficientNet-B0	NPU INT8	No executable configuration

E. Graph-Optimization Sensitivity

Fig. 4 shows optimization speedups (extended vs. disabled ORT). Ratios cluster close to 1.0 on all three backends. Across the evaluated GNNs, model-averaged iGPU ratios range from approximately 0.995 $\times$ to 1.025 $\times$, and NPU ratios range from 0.980 $\times$ to 1.018 $\times$. These small, mixed changes do not support attribution to a specific fusion mechanism.

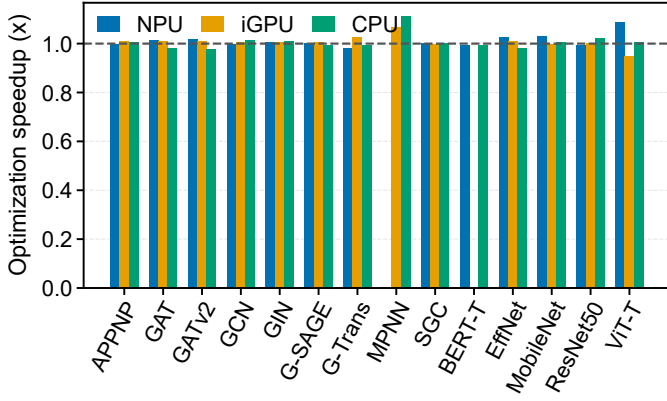


Fig. 4. ORT optimization ratio by device, averaged across datasets and available precisions for each model. Values near 1.0 indicate limited sensitivity to the selected graph-optimization setting.

F. Graph-Size Scaling and Static-Shape Compilation

In the APPNP FP32 size sweep at a fixed density of 7 edges/node, NPU latency increases from 5.92 ms at 512 nodes to 36.03 ms at 8192 nodes ($6.1\times$ latency for $16\times$ as many nodes). Each point is a separately compiled fixed-shape graph, so this experiment characterizes size scaling across compiled shapes rather than dynamic-shape execution within one compiled model.

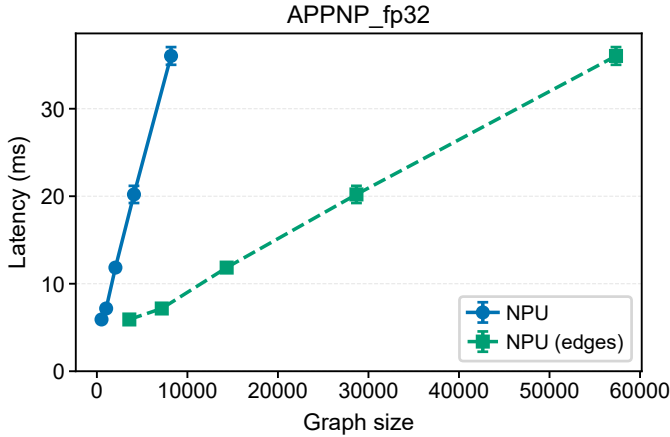


Fig. 5. APPNP FP32 latency scaling on NPU across graph sizes (nodes N and edges E at 7 edges/node). Solid line shows scaling by nodes; dashed line shows scaling by edges. Error bars show run-to-run standard deviation; each point represents a separately compiled fixed-shape graph.

G. Package-Power Analysis

Table V shows package-level power collected via SoCWatch (`-f sys -t 30 -r sum`) and a latency-derived energy estimate. The iGPU draws slightly higher package power than the CPU for GCN (12,482 mW vs. 11,629 mW, $+7.3\%$) but reduces latency (6.94 ms vs. 7.73 ms); their corresponding heuristic energy estimates are 86.6 and 89.9 mJ. For MPNN, CPU and iGPU package power (9,357 vs. 9,473 mW) and latency (20.23 vs. 22.03 ms) are similar. For GCN on CPU, INT8

reduces package power (11,629 to 9,675 mW, -16.8%) with stable latency (7.73 to 7.59 ms, -1.8%), reducing the latency-derived estimate by 18.4%. Conversely, MPNN CPU INT8 increases latency to 32.94 ms ($+63\%$ over FP32: 20.23 ms) and the estimate to 301.1 mJ ($+59\%$ over FP32: 189.3 mJ), despite a slight power decrease (9,139 vs. 9,357 mW). These estimates are model-dependent and are not direct active-inference energy measurements. NPU power-rail isolation was unavailable under the current PMT interface.

TABLE V
PACKAGE POWER AND LATENCY-DERIVED ENERGY ESTIMATE

Model	Config	Avg Power (mW)	Latency (ms)	Energy/Inf. (mJ)
GCN	CPU FP32	11,629	7.73	89.9
GCN	CPU INT8	9,675	7.59	73.4
GCN	iGPU FP32	12,482	6.94	86.6
MPNN	CPU FP32	9,357	20.23	189.3
MPNN	CPU INT8	9,139	32.94	301.1
MPNN	iGPU FP32	9,473	22.03	208.7
MPNN	iGPU INT8	9,161	32.48	297.6

Latency is the mean time over 100 iterations $\times$ 3 repeats across three OGB-derived inputs. Energy/inf. is a heuristic product of package-level average power from a 30 s SoCWatch window and separately measured mean latency. The collection includes background activity and portions of the benchmark workflow; values are not direct measurements of active inference energy. NPU power isolation was unsupported by the PMT interface.

V. DISCUSSION

A. Why INT8 Underperforms for GNNs on NPU

The limited INT8 gains are consistent with the combination of irregular indexing, conversion/runtime overheads, and compiler coverage. Precision reduction lowers arithmetic cost but does not remove sparse Gather/Scatter traffic; the exported GNNs also contain $2\text{--}4\times$ more shape-manipulation nodes than the vision models. Indirect indexing can restrict fusion opportunities [23], although this experiment does not isolate individual compiler passes.

Outcomes are model-specific: MobileNetV2 INT8 exhibits CPU-like timing, no retained native-NPU result is available for MPNN, and three INT8 graphs fail to produce executable configurations. These exceptions make retained device-assignment evidence essential; timing similarity alone is not treated as proof of CPU placement.

B. Platform Maturity Assessment

Under OpenVINO 2024.1, the Meteor Lake NPU provides its largest CPU-relative gains for selected dense models ($4.5\text{--}11.4\times$), while the iGPU leads most evaluated GNN measurements. Graph-optimization ratios remain close to 1.0 on both iGPU and NPU. Memory-system behavior and compiler mapping are plausible contributors to the observed model dependence, but this benchmark does not isolate their effects.

C. Comparison with Prior Work

Custom GNN accelerators like HyGCN [3] ($10^3\times$ over CPU), EnGN [1] ($1.8\times 10^3\times$ over CPU), and GRIP [4] ($17\times$ over CPU) rely on hardware-native sparse engines and dedicated memory hierarchies. By contrast, the Meteor Lake NPU provides $0.9\text{--}1.2\times$ acceleration for the evaluated GNNs over the CPU. These published speedups are not directly comparable because platforms, datasets, and baselines differ; they illustrate the distinct optimization targets of sparse GNN ASICs and commodity client NPUs. This distinction is consistent with observations in the wider edge AI landscape [19].

D. Limitations

Single-Platform Scope: All measurements were collected on a single Intel Core Ultra 5 125H processor (4P+8E+2LPE cores, 7 Xe-cores iGPU, one NPU tile). Results may vary across Meteor Lake configurations with different core/memory setups (LPDDR5x vs. DDR5) and subsequent architectures (Lunar Lake, Arrow Lake) due to differences in memory bandwidth.

Statistical Reporting: Latencies represent arithmetic means over 100 iterations $\times$ 3 independent runs, with sample standard deviations and Student-*t* intervals computed across the three run means. Formal paired hypothesis tests were not conducted. Small latency variations, such as the $\sim 6\%$ CPU-NPU gap for several GNNs, are not assessed for statistical significance, and rankings may be affected by run-to-run laptop environment variability (DVFS, thermal throttling).

Power Measurement Constraints: SoCWatch measurements (Table V) cover only two models (GCN, MPNN) and two backends (CPU, iGPU). NPU power measurement was precluded by PMT hardware counter definitions, which expose NPU temperature and bandwidth (VPU_MEMORY_BW) but lack direct power rail counters, causing NPU-targeted sessions to produce only raw ETW files without power data.

Energy Estimation: The latency-derived estimate (Avg Power $\times$ Mean Latency) combines package power measured over a 30s collection window with separately measured steady-state latency. It does not isolate active inference energy from background activity, compilation, warm-up, or other workflow phases and should not be interpreted as a direct joule-per-inference measurement.

Compiler, Input, and Quantization Constraints: The benchmark uses fixed 2,708-node, 10,000-edge inputs consistent with static-graph NPU compilation; it does not execute the complete OGB graphs. The mixed NNCF/ONNX Runtime conversion pipeline and absence of post-quantization task-accuracy evaluation limit controlled quantizer comparisons. Conversion provenance was not retained in the result tables, and no executable INT8 configuration was produced for GAT, GATv2, or EfficientNet-B0.

Workload Representativeness: The selected GNN architectures include established models that remain common reference points for systems research, and the OGB datasets provide standardized, real-world graphs with widely varying size and density. Nevertheless, the suite does not cover newer large

graph transformers, heterogeneous graphs, temporal graphs, or recommendation workloads. The results should therefore be interpreted as a characterization of representative foundational inference kernels on this client NPU, not as a comprehensive ranking of current GNN applications.

Software Stack Versioning: Evaluation was limited to OpenVINO 2024.1 and ONNX Runtime 1.18. Since the OpenVINO NPU plugin is under active development, subsequent driver and software updates may expand operator coverage and improve GNN performance metrics.

VI. CONCLUSION

Across 14 models and fixed-shape inputs derived from three OGB datasets, the Meteor Lake NPU provides $4.5\text{--}11.4\times$ CPU-relative speedups for selected dense FP32 workloads but little latency benefit for most evaluated GNNs. INT8 outcomes include marginal gains, unavailable executable configurations, a CPU-like timing anomaly, and a $2.2\times$ SGC regression; NPU power could not be isolated. The iGPU achieves the lowest latency for most tested GNNs, while the preferred backend for dense models remains model-dependent. These results motivate benchmarking each deployment artifact on its target backend and retaining device-assignment evidence for anomalous configurations. The suite is available at <https://yusufarbc.github.io/intel-npu-gnn-benchmarking/>.

ACKNOWLEDGMENT

Google Gemini was used to provide limited code-scaffolding suggestions for portions of the evaluation scripts described in Section III and language-editing suggestions for Sections I, II, IV, and V. The authors independently reviewed, tested, verified, and revised all AI-assisted material and retain full responsibility for the code, analysis, interpretation, and manuscript. The authors executed the experiments and verified the reported numerical results and figures.

REFERENCES

- [1] S. Liang, Y. Wang, C. Liu, L. He, H. Li, D. Xu, and X. Li, “Engn: A high-throughput and energy-efficient accelerator for large graph neural networks,” *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.
- [2] A. Auten, M. Tomei, and R. Kumar, “Hardware acceleration of graph neural networks,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, July 2020, pp. 1–6.
- [3] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, “Hygcn: A gcn accelerator with hybrid architecture,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.
- [4] K. Kinningham, P. Levis, and C. Ré, “Grip: A graph neural network accelerator architecture,” *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.
- [5] Z. Qu, D. Niu, S. Li, H. Zheng, and Y. Xie, “Tt-gnn: Efficient on-chip graph neural network training via embedding reformation and hardware optimization,” in *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct 2023, pp. 452–464.
- [6] W. Gomes, S. Morgan, B. Phelps, T. Wilson, and E. Hallnor, “Meteor lake and arrow lake: Intel next-gen 3d client architecture platform with foveros,” in *2022 IEEE Hot Chips 34 Symposium (HCS)*, Aug 2022, pp. 1–40, doi: <https://doi.org/10.1109/HCS55958.2022.9895532>.

- [7] Intel Corporation, “Heterogeneous ai powerhouse: Unveiling the hardware and software foundation of intel core ultra processor for the edge,” Intel Corporation, White Paper Document 817736-1.0, 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/817736/heterogeneous-ai-powerhouse-unveiling-the-hardware-and-software-foundation-of-intel-core-ultra-processors-for-the-edge.html>
- [8] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations*, 2017.
- [9] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, “Simplifying graph convolutional networks,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6861–6871.
- [10] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 1025–1035.
- [11] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *International Conference on Learning Representations*, 2019.
- [12] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [13] S. Brody, U. Alon, and E. Yahav, “How attentive are graph attention networks?” in *International Conference on Learning Representations*, 2022.
- [14] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, “Masked label prediction: Unified message passing model for semi-supervised classification,” in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2021, pp. 1548–1554.
- [15] J. Gasteiger, A. Bojchevski, and S. Günnemann, “Predict then propagate: Graph neural networks meet personalized pagerank,” in *International Conference on Learning Representations*, 2019.
- [16] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 1263–1272.
- [17] S. Abadal, A. Jain, R. Guirado, J. López-Alonso, and E. Alarcón, “Computing graph neural networks: A survey from algorithms to accelerators,” *ACM Comput. Surv.*, vol. 54, no. 9, Oct. 2021.
- [18] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, “Ai and ml accelerator survey and trends,” in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, Sep. 2022, pp. 1–10.
- [19] R. Singh and S. S. Gill, “Edge ai: A survey,” *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023.
- [20] OpenVINO Toolkit Core Team, *OpenVINO 2024.1.0 Release Notes*, OpenVINO Toolkit, 2024. [Online]. Available: <https://docs.openvino.ai/2024/about-openvino/release-notes-openvino.html>
- [21] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [22] S. A. Taylor, J. Fernandez-Marques, and N. D. Lane, “Degree-quant: Quantization-aware training for graph neural networks,” in *International Conference on Learning Representations*, 2021.
- [23] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, “Dnnfusion: accelerating deep neural networks execution with advanced operator fusion,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898.
- [24] Z. Zhang, “Unified operator fusion for heterogeneous hardware in ml inference frameworks,” in *2025 6th International Conference on Big Data & Artificial Intelligence & Software Engineering (ICBASE)*, July 2025, pp. 246–250.
- [25] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, B. Bilangar *et al.*, “Mlperf inference benchmark,” in *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.
- [26] T. Baruah, K. Shivdikar, S. Dong, Y. Sun, S. A. Mojumder, K. Jung, J. L. Abellán, Y. Ukidave, A. Joshi, J. Kim, and D. Kaeli, “Gnnmark: A benchmark suite to characterize graph neural network training on gpus,” in *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, March 2021, pp. 13–23.
- [27] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, “Open graph benchmark: Datasets for machine learning on graphs,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 22 118–22 133, 2020.
- [28] Intel Corporation, “Intel Core Ultra 5 Processor 125H Specifications,” 2026, accessed: September 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/236848/intel-core-ultra-5-processor-125h-18m-cache-up-to-4-50-ghz/specifications.html>
- [29] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [30] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.
- [31] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 6105–6114.
- [32] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.
- [33] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, “Well-read students learn better: On the importance of pre-training compact models,” *arXiv preprint arXiv:1908.08962*, 2019.